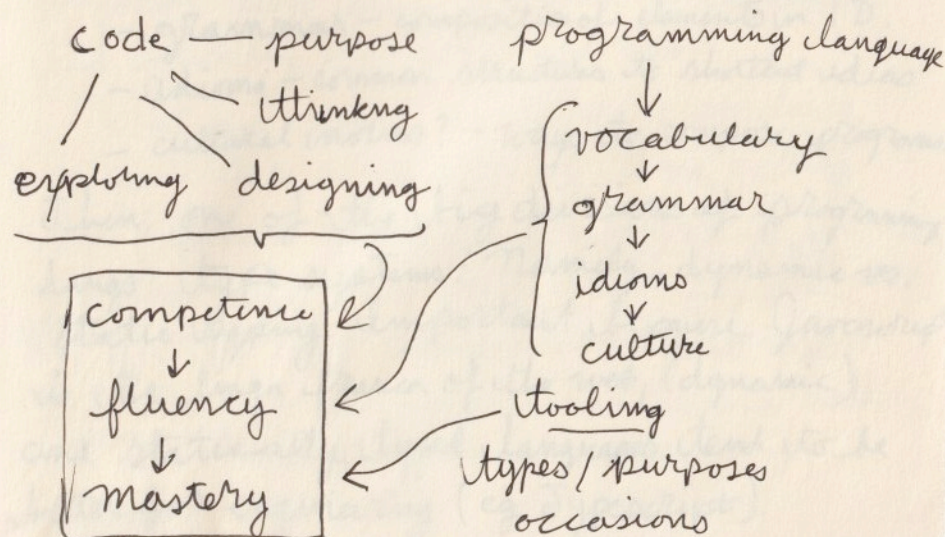


6/21/26

Combining CF1 (intro to programming) and CF2 (data structures & algorithms). This can be a coherent 1-2 round of coding, computational design & engineering, ~~and~~ all situated in a world undergoing a upheaval. We have: code — learn to work with details. Learn the language of computation (which involves but is not entirely made of code) — some math, and the ability to begin thinking, designing, exploring, ~~and~~ convincing in that language. Can follow the same trajectory as a spoken language: learn words, grammar, idioms, cultural norms.



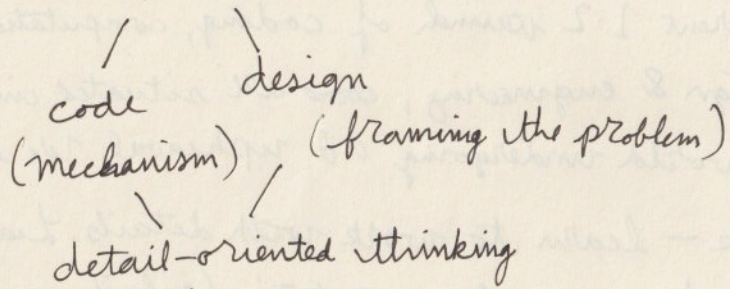
The Writing Fields®

This is an original Flexbook technology product.
All genuine Flexbook binding products have the holographic label.
Patent No PCT/GR2012/000024
Made in Greece with care and sustainable practices.

www.twf.gr

purpose

computation



fluency

mastery

I'm writing an interactive book for CF1 and CF2. So far I have a good start on the software to make this happen. I am still fuzzy (fuzzy?) on the structure but I do know I want it to be oriented around computation as a medium for thinking & design, more than a typical "how to software engineer" tract.

Language is a nice starting point, as everybody has an intuitive awareness of what that entails. From there I can introduce a programming language and draw parallels with natural language:

- syntax - atomic elements, "if", "function", ...
- semantics - meaning of elements
- grammar - composition of elements in 1D.
- idioms - common structures to shortcut ideas
- cultural norms? - ways to compose programs.

Then, one of the big divisions of programming langs: type systems! Namely, dynamic vs. static typing. Important because Javascript is the lingua franca of the web, (dynamic), and statically typed languages tend to be better for engineering (eg Typescript).

Language



dynamic



static

computers are dumb.

goals are elusive

&

take N tries to get right (enough)

~~do~~ details → big picture → fluency → mastery



intuitive control

I think that sets the stage for answering questions about "why" a student would care, or be asked, about learning to code. artificial intelligence will be a big gorilla in the room, and I have to address that early. At the same time, I need to avoid being drawn into the "just use AI" trap that is so pervasive in industry.

So in the intro I have to address AI and its role in learning as distinct from its role in practice, so I do not have to repeat the same old talk all the time.

PROGRAMMING LANGUAGES

Main thing: computer seems smart, but really has no clue. Getting the details right is up to you. It has no idea about intentions. `blue = 17` is a perfectly valid thing to write in Javascript but probably not what you want.

"What you want" is hard to pin down, so you'll take a number of turns getting there. So - get good with the medium so you can get there with less pain & free up your mind to see the big picture. Fluency lets you see through the details. That leads to mastery, and the medium becomes an instrument of intuitive control

DETAILS (the building blocks of code)

- syntax - mistakes wreck things, so syntax highlighting and formatting are your friends
- variables to hold data
 - declare - assign - read - re-assign (initialize)
- function calls - console log to start with, then use p5 graphics (line, circle, rectangle, colors)
- persistence & sequencing
 - set color to red
 - draw a circle
 - set color to blue
 - draw a line

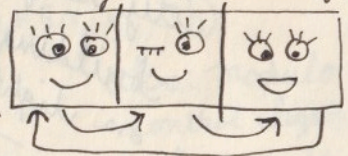
} the earlier calls ~~and~~ set up later calls - order matters

homework - self "portrait"
or something like that.
still life



NOW YOU HAVE JUST ENOUGH
TO MAKE STATIC IMAGES

- kinds of variables - text, booleans, numbers, there are more but we'll get there later.
 - conditionals - if - statements
 - events from the runtime - mouse movement/clicks, keyboard. Use this to make your still life animated and stateful.
 - organize this better with functions: eyeOpen, eyeClosed, mouth, mouthGrin
- or - parameters: eye(isOpen) → eye(leftOrRight, isOpen)
mouth(isGrin)



homework
interactive portrait

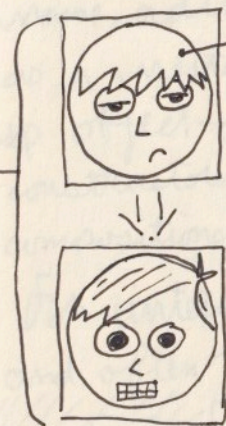
loops

function params.

homework

city skyline w/ stars, moon,
buildings. Keyboard to turn
on lights using modulo.
use variables like building 1,
building 2, etc.

So now we have enough to make a stateful,
interactive scene. Next homework!



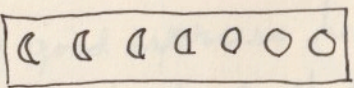
hold mouse over a feature: hair, eyes
(left & right), nose, mouth and have
it highlight it somehow.

* while hovering, press keys to change
its appearance. Eg different hair style,
color, eyes blink, look at different
spots, mouth smiles, etc.

Repetition. Repetition. Repetition. Repetition.

Now introduce repetition in a few ways:

for-loops, while-loops, do-while-loops.



Good for repeating a
pattern identically, or
with variation

Pair this with function parameters to write
cleaner, more organized code.

```
for (i=0 to 10) {  
  moon( phase: i/10, x: i*50, y: 0 )  
}
```



also: modulo to
control lights
without needing to
store individual vals
in a list - we will
get to that.

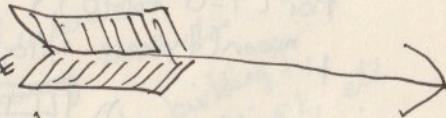
{ objects } classes

dot notation



Random Numbers

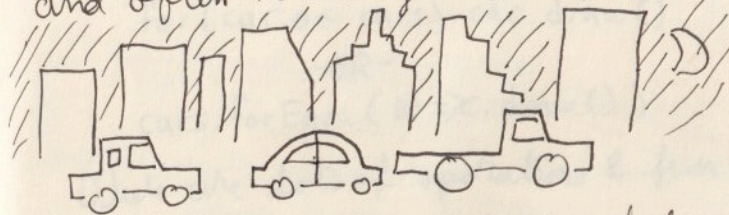
HOMEWORK
CITY SCENE
ANIMATION



OBJECTS?

This can be about simple groupings of data (first/last name, address, etc) or hierarchies (put address as a nested property) or (eventually) straight up object-oriented programming with constructors & encapsulation, abstractions and compositions, and other fancy words as well. (CLASSES!)

For interactive systems this is really useful and often models your domain nicely.



Syntactically, dot notation helps to drive the "ownership" of data.

A good exercise here is to make a few different kinds of objects that can render differently.

Pair this with random numbers & you can get groovy effects. The thing above has car and building object types - maybe add stars.

EMOTION p5 (and every other graphics lib) supports animation, such as by polling a clock, or looking at a frame counter.

Homework: use the city/car exercise above, but use time to animate it. Cars should move forward, and lights in windows should flip on/off without inducing seizures.

Lists.

buildings = makeBuildings(10)
cars = makeCars(30)

HW

city scene w/ lists
for buildings & cars.

HW geometry
face editor

[LISTS]

making sequential lists of items helps
break out of manual management. Instead
of car1, car2, car3, etc you can have
cars = [millions of cars go here], and this
is more than just a convenience. It lets you
work with variable numbers of things, and
operate on them as a collection:

```
for (car of cars) car.draw()
```

-OR-

```
cars.forEach(c => c.draw())
```

There are lots of operations & fun shortcuts.
push, pop, split, join, length, all the
iteration functions, etc.

With this you can rewrite the city/car homework
using lists. Add keyboard support for adding,
removing cars and buildings, and use lists instead
of manual management.

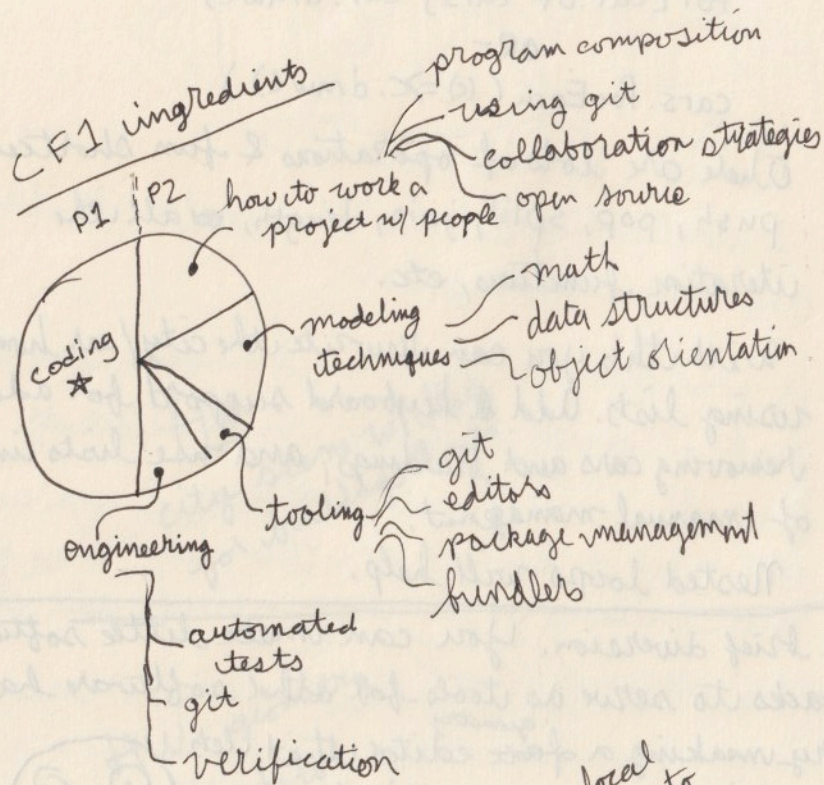
Nested loops will help.

A brief diversion. You can make little software
hacks to serve as tools for other software hacks.

Try making a ^{geometry} face editor that lets
you (a) select elements, (b) change their
colors/styles, (c) change their geometry
or render mode, and most important,
(d) output the configuration as an object so it
can be used elsewhere. Color/style might be too much.



HW "Nuance"
 pick some nuance topics &
 build something around them.
 - Breakout game
 - Traffic sim
 - Whimsical animation
 - Interactive graphic novel.
 - Recursive tree
 - Boids



P1 - all just coding for first project
 P2 - group project for local work - must use git to collaborate.
 P3 - your choice

NUANCE.

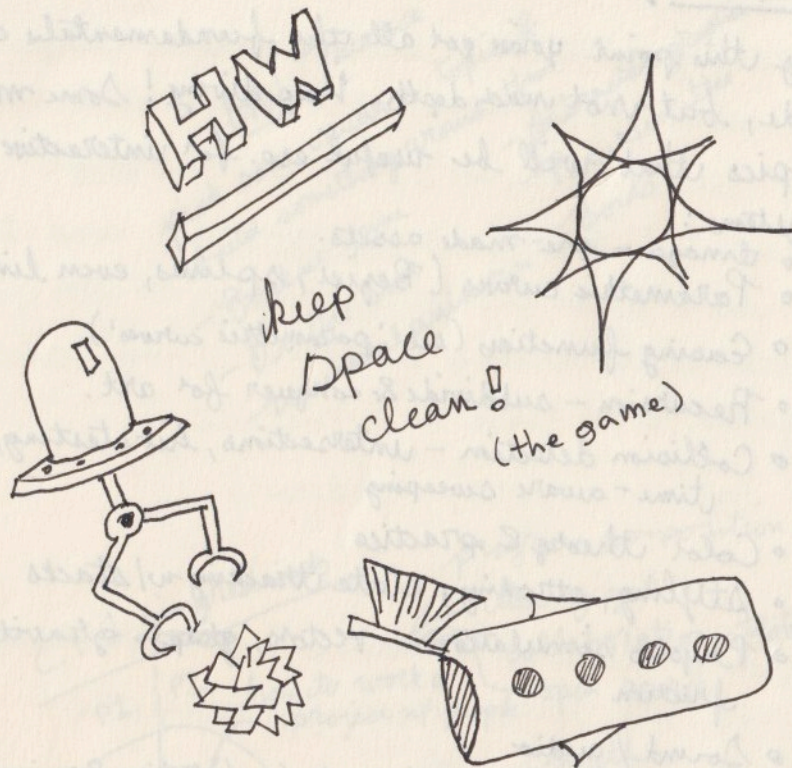
By this point you've got all the fundamentals of code, but not much depth. Keep diving! Some more topics that will be useful esp. for interactive systems:

- Images - pre-made assets.
- Parametric curves (Bezier, splines, even lines!)
- Easing functions (see 'parametric curves')
- Recursion - subdivide & conquer for art.
- Collision detection - intersections, hit testing, time-aware sweeping
- Color theory & practice
- Styling, stroking, state tracking w/ stacks
- Physics simulations - vectors, ~~graphs~~ gravity, friction
- Sound / audio
- Type / typography / text (w/ styling & animation)

There are lots of good exercises hiding here. combine a few of them to make something fun.

Some of that will be doable in a browser editor, but you get more mileage from a local editor & tooling.

- Local editor
- CLI
- git
- Javascript engineering topics
 - libraries
 - browser & file structure
 - node & npm
 - bundlers



Sears christmas
catalog 1983 → so inspiring

To get used to local development, you need to get used to not having every answer available to you. Old fashioned web search and new fangled A&C can likely get you back on the road after you end up in the ditch.

But there should be some homework here too so there's something to work towards. A collaboration would give a good reason to break out of the in-browser editor and remove the training wheels.

Make a game with one other person. A should provide starter code. You will need to figure out how to divide the labor:

- physics
- game play logic
- graphics
- sound
- playtesting
- level design
- music
- story
- dialog
- so much more...

keep in mind you can and probably will need to change your mind after you start. No plan survives first contact with the enemy,

★ The deliverable will include the github link, which must have code and a writeup of how it went.

HW

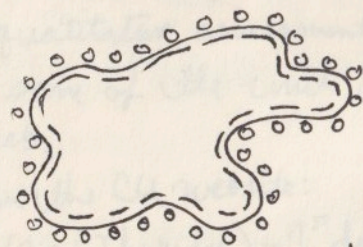
d show a bunch of pictures or drawings, and your job is to write (a) plain english and (b) code that replicates the original.

This is a writing assignment

"what do you see?"

Inspired art comes from someplace - maybe a view on a hike? or photograph? Or your imagination? Maybe it comes from many places over time. Regardless, a main ingredient to the craft is in translating what you "see" into code. This means you must be able to articulate what it is that you see in your mind into another format. And in doing so, you must confront the vision at a level of detail that you might find uncomfortable.

This is a good thing - it means you are learning. Like, what do you see like?



How would you describe this in plain English? If I showed one or six more drawings, do you think you could describe

the general features? Move from the particular to the population? If you can do this well enough in natural language, you can (or, will be able to) do it in code. It can be useful for reflecting your writing back to you. But if you let it think for you, you will lose in the long term.

New course structure

CF1 should tee students up for CF2 for sure, but also other technical classes in Atlas. git is the main thing but local editors will help with writing for Arduino etc. So I want them to be tested on that.

Last time the CF2 students gave feedback on the CF1+2 sequence, many said they want more practice, and even quizzes.

They did like the weekly cycle, where it was clear when homework was due. I think the unit test approach by itself is too cheatable - a coding assistant will just do it for them & they learn nothing. So quizzes and quantitative assessment is important, & I will keep some of the unit test stuff as well as projects.

From the CU website:

My class is M/W 9:30-11

8/20 (Thursday) - 1st day of classes

9/7 (Monday) - Labor Day

11/23-27 (M-F) - Fall Break

12/4 (Friday) Last day of class

- cancel 8/21 Recitation (no point)
- also cancel 12/14
- + make HW due on Monday (or Wednesday after labor day)
- + Quizzes due Wednesday

Quizzes are on preceding Wed/Monday
 Homework varies - coding generally (p5 sketch, later git repo), can be writing

3 - Projects

... all of that is aspirational.

project 1 - solo

project 2 - group, must be local editor & use git.

project 3 - solo or group, you choose what/how.

Language. Make static things

Interactivity. Make live things (conditionals)

Animation. Objects. Repetition

MTWTFSS

AUG 20 21 22 23

	24	25	26	27	28	29	30	① - 27 class sessions
SEP	31	1	2	3	4	5	6	- 14 weeks
labor day	8	9	10	11	12	13	14	- last week (Nov 30, Dec 2) for presentations
	15	16	17	18	19	20	21	- Recitations are Fridays
	22	23	24	25	26	27	28	- 13 Recitations
OCT	29	30	1	2	3	4	5	
	6	7	8	9	10	11	12	
	13	14	15	16	17	18	19	
	20	21	22	23	24	25	26	
NOV	27	28	29	30	1	2	3	
	4	5	6	7	8	9	10	
	11	12	13	14	15	16	17	
	18	19	20	21	22	23	24	
	25	26	27	28	29	30	1	
	2	3	4	5	6	7	8	
	9	10	11	12	13	14	15	
	16	17	18	19	20	21	22	
full break	23	24	25	26	27	28	29	
DEC	30	1	2	3	4	5	6	
	↑	↑	↑					
	HW Due	Quizzes	Recitations					

maybe swap weeks 8 & 9
2

Weeks!

- Language
- [self portrait] Interactivity
- [interactive portrait] Animation
- [city scene] Lists, Objects, Tooling
- [editor] Nuance topics
- [nuance] Tooling: Git, CLI, editors
- Project 1
- [Space Junk] Seeing, Modeling
- [what do you see?] JS tooling npm, node
- [git space junk] SPT: data structs,
- Project 2
- [AABB] web & object
- [web page] UX & AI
- Project 3

git/editor/cli:

The space junk assignment was a ~~text~~ browser based edit (p5.js) but we need practice getting projects up on Github. So this week is about porting your earlier code + assets to a Repo.

Several parts to this: all the code and assets have to be there. It has to be accessible to instruction staff. (So, permissions ~~are~~ are ok) We should be able to clone each Repo & npm i, and run it. The app should render and plausibly be the thing they made previously. It should have a README.

At this point I've covered everything that CF1 must cover. But to tie them up for other classes like web, object, and data structures, we need more.

HW: hand-rolled web
site - nicely formatted
and styled story of
your CF1 experience

HW: AABB 
adopted from 2270

← web → How does a js app even run in a browser?
HTML, CSS, DOM, scripts & execution.

object → how does code execute on a device? OS, VM,
or sandbox models of computation. memory, I/O,
networking

← data structures → how you model data can have a
huge effect on performance; frame rate, junk,
or what is even possible to include in a game or app

UX → the user is part of the system, and in
general, computation only matters if it is in
service to them

AI → Global topic du jour. How and why it works,
& what you need to understand as it evolves.

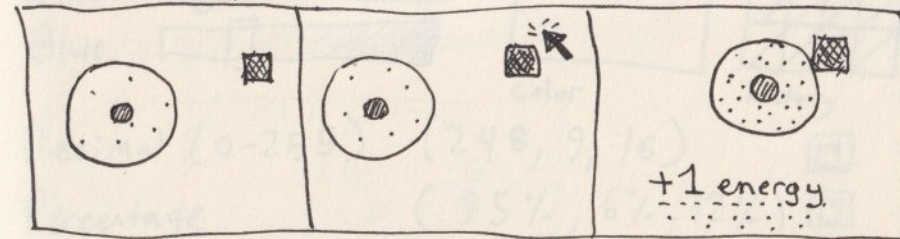
Book vs Homework vs Slides vs Quizzes

6/27/26

Last day in Hawaii
;-

I don't want the book to depend on slides or homework - I want it the other way around. I don't yet know if I will have a way to connect the book's quiz feature back to canvas, or how that could work. Prof need to build canvas quizzes separately. Slides fall out of the book, can maybe even have Ad do that... perhaps a plain HTML and CSS slide approach?

Either way, the book comes first. The slides and quizzes come from that, and the homework can be mentioned - though maybe I have several homework options there to keep variety up.



Static image - a blue blob and piece of food

interactivity - click and the blob gets a destination

animation - the blue blob moves and eats the food! Show text & change appearance; remove the food after it is eaten. (27)